GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

Lamachaur, 18, Kaski, Nepal

LAB REPORT OF

Agile software development

Lab3:Implementation of Different Types of Software Testing Using the Jest Framework.

Submitted By:	Submitted To:
Sandip Aryal Roll no: 35	Er.Rajindra Bdr. Thapa

OBJECTIVE

To implement and perform unit testing on a JavaScript calculator application using the Jest framework and to study different software testing tools and techniques such as Test Case Generation, JUnit, Selenium, and Load Testing.

TOOLS AND TECHNOLOGIES

- **Nodejs :** A JavaScript runtime environment needed to execute the calculator application and test scripts outside a web browser.
- **npm :** The package manager used to install Jest and manage project dependencies.
- **Jest Framework**: A JavaScript testing framework used to define, execute, and report on unit tests.
- **VS Code / Text Editor**: For writing the source code for the calculator and creating the Jest test suites.
- **Command Line Interface (CLI:** To initialize the project, run the test commands, and view test execution reports.

THEORY

Software testing is the process of evaluating and verifying that a software product or application does what it is supposed to do. Different levels of testing exist, ranging from unit testing (isolated components) to system and acceptance testing.

Jest is a modern, zero-configuration testing framework primarily used for JavaScript and Node.js applications. It comes with built-in assertion libraries, mocking capabilities, and test runners, making it ideal for Test-Driven Development (TDD).
Alongside Jest, this experiment explores several other critical tools and concepts in the broader scope of software quality assurance:

Test Case Generation:The process of designing test inputs and expected outputs to ensure thorough code coverage. Techniques include Equivalence Partitioning (testing inputs in groups) and Boundary Value Analysis (testing edge cases, such as minimum/maximum values).

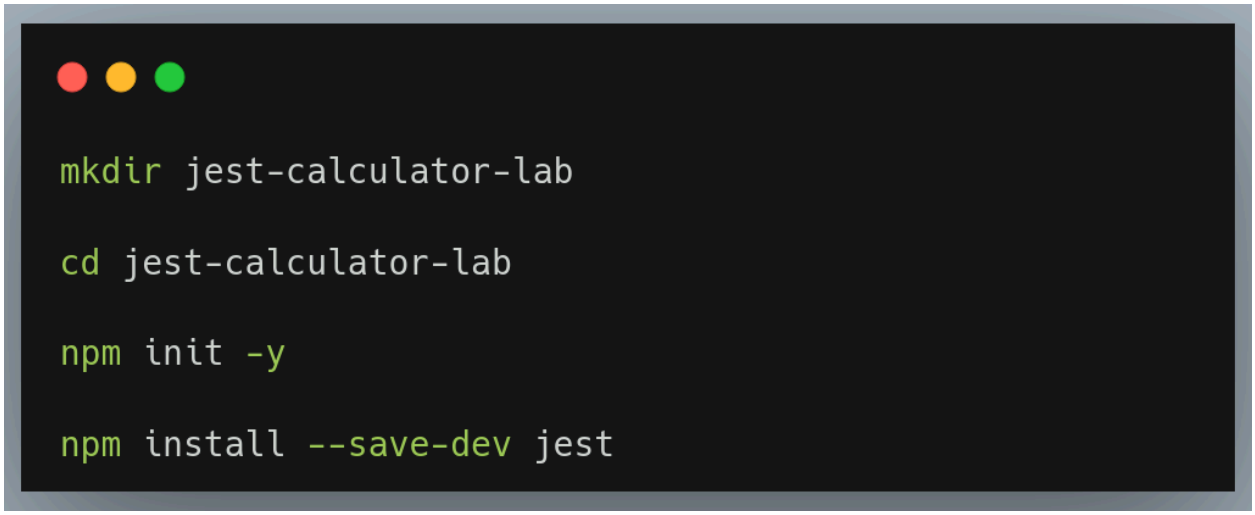
JUnit:A widely used testing framework for Java applications. Unlike Jest (JavaScript), JUnit is used for Java-based back-end or Android development, following a similar unit testing philosophy but with different syntax.

Selenium: A powerful open-source tool specifically designed for automated UI and web application testing. While Jest executes JavaScript logic in the background, Selenium automates real web browsers (like Chrome or Firefox) to simulate user clicks, form inputs, and navigation.

Load Testing: A non-functional testing technique used to simulate real-world usage loads (e.g., thousands of concurrent users) to determine how a system behaves under heavy stress. Tools like JMeter or LoadRunner are commonly used for this.

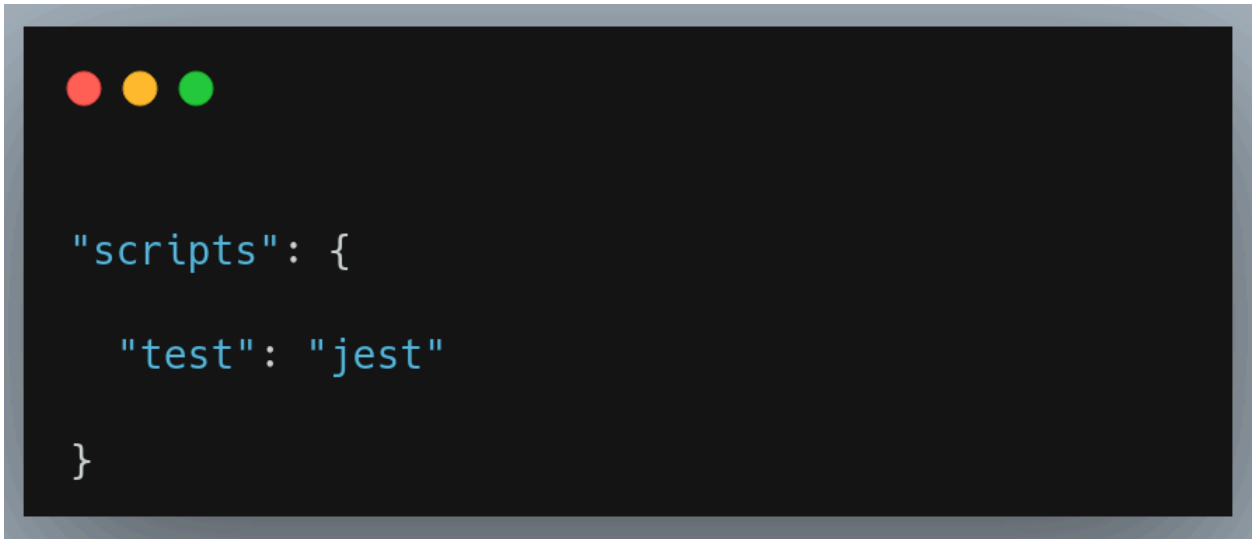
PROCEDURE

1. **Environment Setup:**
   Create a new project directory and initialize it with npm. Install Jest as a development dependency, and update the `package.json` file to enable Jest testing scripts.



```
mkdir jest-calculator-lab

cd jest-calculator-lab

npm init -y

npm install --save-dev jest
```

Modify the package.json scripts block:



```
"scripts": {

  "test": "jest"

}
```

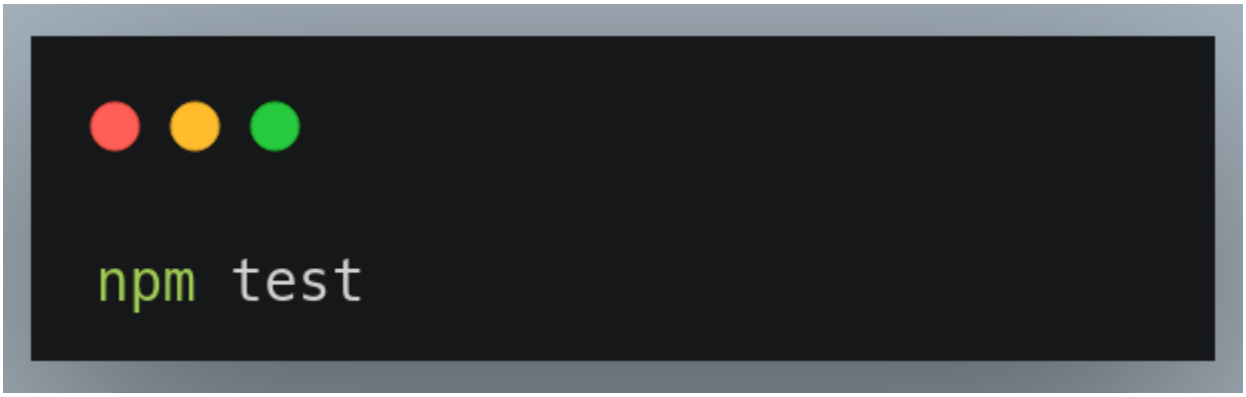
2. **Writing the Application Logic**
   Create a source file named calculator.js containing the basic arithmetic functions to be tested.

```
// calculator.js

function add(a, b) {

  return a + b;

}

function subtract(a, b) {

  return a - b;

}

function multiply(a, b) {

  return a * b;

}

function divide(a, b) {

  if (b === 0) {

    throw new Error("Division by zero is not allowed");

  }

  return a / b;

}

module.exports = { add, subtract, multiply, divide };
```

3. **Generating Test Cases and Writing Test Suites**
   Create the test file calculator.test.js. Here, test cases are generated to cover standard functionality, boundary conditions, and expected error handling.

```
const { add, subtract, multiply, divide } = require('./calculator');

// Standard Unit Tests
test('adds 5 + 10 to equal 15', () => {
  expect(add(5, 10)).toBe(15);
});

test('subtracts 10 - 4 to equal 6', () => {
  expect(subtract(10, 4)).toBe(6);
});

test('multiplies 7 * 8 to equal 56', () => {
  expect(multiply(7, 8)).toBe(56);
});

// Boundary and Exception Tests
test('handles division by zero gracefully', () => {
  expect(() => divide(10, 0)).toThrow("Division by zero is not allowed");
});

test('divides 12 by 4 to equal 3', () => {
  expect(divide(12, 4)).toBe(3);
});

// Testing Edge Cases (Boundary Value Analysis)
test('adds negative numbers correctly: -5 + (-3) equals -8', () => {
  expect(add(-5, -3)).toBe(-8);
});
test('multiplies by zero correctly: 10 * 0 equals 0', () => {
  expect(multiply(10, 0)).toBe(0);
});
```

4. **Executing the Test Suite** Run the defined test suite using the Node.js command line. Jest automatically searches for any files ending in .test.js or .spec.js and runs them sequentially.



RESULT

Upon executing the npm test command, Jest scans the project, runs the defined test cases, and provides a comprehensive console report. The output displays a green checkmark for passing tests, alongside a summary of the total tests run, the number passed, and the number failed. If any test fails, Jest provides a detailed diff of the expected versus actual output, allowing for immediate debugging. For this specific calculator implementation, all generated test cases—covering normal operations, negative numbers, zero multiplication, and division-by-zero exceptions—should pass successfully, confirming that the calculator module is mathematically and logically robust.

CONCLUSION

This experiment successfully demonstrated the implementation of unit testing on a JavaScript calculator application using the Jest framework. By generating meaningful test cases based on boundary value analysis and exception handling, the reliability of the basic arithmetic functions was validated.